

Debug Mạng từ A–Z

Hệ thống hóa kỹ năng chẩn đoán mạng

Network Thực Chiến · Series tổng hợp · 11 tập · 7 module

Tại sao cần một series về debug mạng?

Khi mạng có vấn đề, hầu hết kỹ sư làm gì?

- ✗ Google "why is ping slow"
- ✗ Restart service và cầu may
- ✗ "Mình không biết, check lại firewall đi"
- ✗ Hỏi đồng nghiệp mà ai cũng đoán mò

Vấn đề: Không phải thiếu tool — mà thiếu **phương pháp**.

Một kỹ sư mạng giỏi không đoán mò — họ có **methodology**, có **công cụ**, và biết hỏi đúng câu hỏi ở đúng tầng OSI.

Nội dung series

1. **Methodology** — Tư duy debug có hệ thống
2. **OSI Debug Map** — Mỗi tầng, một câu hỏi, một công cụ
3. **7 Module · 11 Tập** — Từ `ping` đến Kubernetes eBPF
4. **Lộ trình học** — Người mới → Kỹ sư nâng cao → K8s Engineer
5. **Kịch bản thực chiến** — 3 scenario ghép nhiều công cụ

Phần 1

Methodology: Tư duy trước, tool sau

Quy trình debug 5 bước

Áp dụng cho **mọi sự cố mạng** — từ ping không tới đến K8s pod mất kết nối:

Bước 1 – XÁC ĐỊNH TRIỆU CHỨNG

"Cái gì bị lỗi? Từ đâu? Đến đâu? Từ khi nào?"

Bước 2 – ĐẶT GIẢ THUYẾT

"Tầng OSI nào có thể gây ra lỗi này?"

Bước 3 – TEST VÀ LOẠI TRỪ

Dùng tool phù hợp, test từng tầng — không nhảy cóc

Bước 4 – TÌM ĐIỂM PHÂN GIỚI

"Từ đây hoạt động, từ đây không → root cause nằm ở giữa"

Bước 5 – XÁC NHẬN VÀ DOCUMENT

Ghi root cause, fix, cách phòng ngừa

OSI Model — Góc nhìn debug thực tế

Tầng	Câu hỏi cần trả lời	Công cụ
L1 — Physical	Cáp có cắm? NIC có up?	<code>ip link</code> , <code>ethtool</code>
L2 — Data Link	ARP resolve được không? MAC đúng không?	<code>arp</code> , <code>ip neigh</code> , <code>tcpdump</code>
L3 — Network	Route đúng không? Ping tới không? Mất gói ở đâu?	<code>ping</code> , <code>mtr</code> , <code>ip route</code>
L4 — Transport	Port có mở? TCP handshake thành công không?	<code>ss</code> , <code>nc</code> , <code>tcpdump</code>
L5–6 — Session/TLS	TLS cert hợp lệ không? Handshake lỗi ở đâu?	<code>openssl s_client</code> , <code>curl -v</code>
L7 — Application	DNS resolve đúng? HTTP trả gì? App lỗi gì?	<code>dig</code> , <code>curl</code> , <code>tcpdump port 80</code>

Quy tắc vàng: Debug từ dưới lên (L1 → L7).
Đừng debug DNS khi `ping` còn không tới được.

Phần 2

Bản đồ 7 Module · 11 Tập

Module 1–2: Connectivity & Transport

Tập	Tool	Mô tả ngắn
01	<code>ping</code>	ICMP, MTU Discovery, TTL Fingerprinting, Flood Ping
02	<code>mtr</code>	Kết hợp ping + traceroute — đọc StDev, phát hiện bottleneck
03	<code>ss</code>	Thay thế <code>netstat</code> — TCP states, port leak, connection tracking
04	<code>netcat (nc)</code>	Dao Thụy Sĩ TCP/UDP — test port, tạo server tạm, debug firewall

Học xong Module 1–2: Kiểm tra được kết nối từ L3 đến L4 mà không cần GUI.

Module 3–5: DNS, HTTP, Packet Capture

Tập	Tool	Mô tả ngắn
05	<code>dig</code>	Query types, trace delegation, debug K8s CoreDNS, flush TTL
06	<code>curl</code>	Headers, auth, TLS cert, redirect chain, timing breakdown
07	<code>tcpdump</code>	Filter syntax, bắt ra file, decode TCP flags, remote capture
08	<code>Wireshark</code>	Remote capture từ server, follow TCP stream, decode protocol

Học xong Module 3–5: Nhìn thấy gói tin thực tế đi từ client đến server.

Module 6–7: Performance & Kubernetes

Tập	Tool	Mô tả ngắn
09	iPerf3	TCP/UDP throughput, jitter, parallel streams, JSON output
10	netshoot	Pod network debug, capture trong container, tcpdump + mtr trong K8s
11	Hubble / Inspektor Gadget	eBPF-based observability, Layer 7 tracing trong K8s

Học xong Module 6–7: Benchmark được hiệu năng mạng và debug được sự cố trong môi trường container/K8s.

Phần 3

Lộ trình học

Chọn lộ trình phù hợp

Người mới bắt đầu — Xây nền vững:

Tập 01 (ping) → Tập 02 (mtr) → Tập 03 (ss) → Tập 05 (dig)

Nắm được 80% sự cố mạng thông thường.

Kỹ sư muốn nâng cấp — Đào sâu hơn:

Tập 04 (nc) → Tập 06 (curl) → Tập 07 (tcpdump) → Tập 08 (Wireshark)

Debug được từ L4 lên L7, nhìn thấy gói tin.

Kỹ sư K8s / Cloud Native — Full stack:

Toàn bộ Module 1–6 → Tập 09 (iPerf3) → Tập 10 (netshoot) → Tập 11 (eBPF)

Debug được mọi tầng — từ NIC đến application trong container.

Phần 4

Kịch bản thực chiến

Scenario A: "Website load chậm, không biết lỗi ở đâu"

Phương pháp điều tra từ dưới lên:

```
# L3 – Có tới không? Latency cao không?  
ping google.com
```

```
# L3 – Bottleneck ở hop nào? Mất gói ở đâu?  
mtr -rw google.com
```

```
# L7 – DNS resolve đúng không? TTL còn bao lâu?  
dig google.com
```

```
# L7 – HTTP layer trả gì? Redirect không? TLS lỗi không?  
curl -v https://google.com
```

```
# L4/L7 – Gói tin thực tế ra sao?  
tcpdump -i eth0 port 80 or port 443
```

Scenario B: "Microservice A không kết nối được Service B"

```
# L4 – Port B có đang listen không?  
ss -tlnp | grep <port>  
  
# L4 – Firewall / NetworkPolicy có chặn không?  
nc -zv <service-B> <port>  
  
# L7 – DNS trong cluster có resolve không?  
dig <service-B>.<namespace>.svc.cluster.local  
  
# L7 – App có trả lời không? Health check ra sao?  
curl http://<service-B>:<port>/health
```

K8s bonus: Nếu vẫn không được → `netshoot` pod, tcpdump trên node.

Scenario C: "Throughput mạng thấp hơn mong đợi"

```
# L3 – MTU có bị cắt không? (1500 – 20 IP – 8 ICMP = 1472)  
ping -s 1472 -M do <host>
```

```
# L3 – Path có packet loss không? Jitter cao không?  
mtr -r <host>
```

```
# L6 – Thực tế đạt bao nhiêu Mbps?  
iperf3 -c <server> -t 30 -P 4
```

```
# L4 – TCP window scaling, retransmit có không?  
tcpdump -w capture.pcap host <server>  
# → Mở Wireshark: Statistics > TCP Stream Graphs
```


Key Takeaways

3 nguyên tắc:

1. **Methodology trước** — Không đoán mò, debug từ L1 lên L7
2. **Tool đúng tầng** — `ping` cho L3, `ss` cho L4, `dig` cho DNS, `curl` cho HTTP
3. **Kết hợp tool** — Sự cố thực tế cần nhiều tool phối hợp, không một tool nào đủ

Bộ công cụ cốt lõi cần nắm:

```
ping   → mtr   → ss   → nc   → dig   → curl   → tcpdump
L3      L3      L4      L4      L7      L7      L3-L7
```

Nắm 7 tool này → xử lý được 95% sự cố mạng gặp trong thực tế.

Bắt đầu từ Tập 01

Network Thực Chiến

"Debug không phải là may mắn — đó là phương pháp."